

ShopSphere

S13 - Payments, Gateway Verification and Idempotency

Customer Portal

Platform-Specific Sprint Planning Document

This document intentionally defines only the behavior and delivery scope of the selected platform. It does not mix implementation details from the other platforms.

1. Sprint Definition

Sprint: S13 - Payments, Gateway Verification and Idempotency

Platform: Customer Portal

Application: ShopSphere

Objective: Implement the customer-facing Payments, Gateway Verification and Idempotency behavior for iOS, Android and Web.

This sprint is complete only when the stated use cases, acceptance criteria and tests pass for this platform.

2. Business Use Cases

ID	Use Case	Actor	Precondition	Main Flow	Expected Result
UC-01	Pay order	Customer	Valid checkout	Create payment, complete gateway flow, verify callback	Order/payment reaches a confirmed server state.
UC-02	Process webhook	Gateway	Authentic webhook	Verify signature and idempotency, update payment	Duplicate/replayed webhook has no duplicate effect.

3. Detailed Implementation Plan

1. Implement payment method selection, gateway handoff, success/failure/cancel/timeout/retry.
2. Show confirmation only after backend payment verification.

4. Data / State / Contract Scope

- No direct database changes. Data is accessed through backend APIs.

API / Integration Contract

- Consume the corresponding /api/v1 feature APIs defined by the backend sprint document.

5. Rules and Expected Behavior

- The platform must rely on the backend as the source of truth for business rules.
- Do not expose stack traces, secrets or internal implementation details to end users.
- Handle loading, empty, validation and error states explicitly.
- Requests that can be retried must be safe from duplicate side effects where the feature requires idempotency.
- Ownership and authorization must be checked server-side for protected data.

6. Acceptance Criteria

- Customer journey is complete on iOS, Android and Web where the feature applies.
- UI consumes backend responses rather than duplicating business calculations.
- Loading, empty, validation and error states are implemented.
- Unauthorized or stale data is handled safely.

7. Test Cases

TC	Scenario	Expected Result
TC-S13-01	Gateway timeout	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S13-02	Duplicate webhook	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S13-03	Forged signature	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S13-04	Payment success then app closes	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S13-05	Debited but callback timeout	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

TC	Scenario	Expected Result
TC-S13-06	Retry after timeout	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S13-07	Duplicate payment request	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S13-08	COD unavailable	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

8. Sprint Exit Checklist

- Implementation completed only for this platform.
- All primary and negative use cases pass.
- API contracts are implemented/consumed as defined.
- Validation and authorization tests pass.
- Loading/empty/error handling is verified where applicable.
- No critical or blocker defect remains.
- Automated tests pass in CI.
- Documentation and contract changes are committed.

9. Out of Scope for This Document

- Implementation behavior of Backend + DB. That behavior belongs in the separate S13 document for that platform.
- Implementation behavior of Admin Portal. That behavior belongs in the separate S13 document for that platform.